

Seed Computing Research Division | Technical
Whitepaper

Jasper 2.0

*An Agent Mesh and Orchestration System for the Seed
Computing Paradigm*

Leon | Seed Computing Research Division | July 2026

Version: 2.0.1 | Classification: Public Release |
Document ID: SC-WP-2026-007

Abstract

The proliferation of multi-agent artificial intelligence systems has exposed fundamental limitations in centralized orchestration paradigms: brittle routing logic, opaque decision chains, inability to enforce safety constraints at runtime, and a static view of agent identity and capability. Jasper 2.0 addresses these limitations as a comprehensive *agent mesh and orchestration layer* designed expressly for deployment within the Seed Computing paradigm — a computing philosophy that treats intelligence, computation, and state as living, mobile, composable entities called *seeds* rather than fixed, location-bound processes.

This whitepaper presents the architecture, mathematical foundations, and operational semantics of Jasper 2.0. Its principal innovations include: (1) a **deterministic routing model** grounded in the Unified Field Equation (UFE), which collapses agent capability vectors, trust tier coefficients, contextual entropy, and load factors into a single, auditable routing scalar; (2) a **capability delegation protocol** with scoped, revocable token chains enabling safe sub-

task handoff across agent boundaries; (3) the **Invariant Engine**, a typed constraint-enforcement subsystem providing pre-, mid-, and post-execution safety verification; and (4) seamless integration with the **Aurora Runtime** execution substrate via the Teleportation Engine, which enables live agent state migration across heterogeneous runtime nodes with full integrity verification.

Through formal analysis, three detailed domain-specific orchestration examples — spanning healthcare, legal compliance, and enterprise automation — and a treatment of broader applications, this paper demonstrates that Jasper 2.0 constitutes a significant advance in the engineering of safe, scalable, and semantically rich multi-agent systems operating under the Seed Computing model.

Keywords: agent mesh, multi-agent orchestration, Seed Computing, deterministic routing, capability delegation, invariant safety, Aurora Runtime, Teleportation Engine, Unified Field Equation

1. Introduction

The emergence of large language model (LLM)-based agents has accelerated the deployment of AI systems capable of reasoning over open-ended tasks. Yet individual agents, however capable, remain fundamentally constrained by their context windows, domain knowledge, and inability to coordinate with peer systems in a principled, verifiable manner. Monolithic AI architectures — those in which a single agent model is expected to handle the full spectrum of a complex workflow — exhibit well-documented failure modes: they degrade gracefully under distributional shift but catastrophically under out-of-distribution task demands, accumulate error through long reasoning chains without external verification, and provide no native mechanism for safety boundary enforcement during mid-task execution.

Multi-agent architectures have emerged as the natural response to these limitations. By distributing cognition and capability across a network of specialized agents — each with a defined domain, permission scope, and behavioral contract — it becomes possible to construct systems that are simultaneously more capable, more auditable,

and more robust than any single-agent solution. However, the orchestration of such networks introduces its own class of problems: how should tasks be routed between agents? How can trust relationships be expressed and enforced at runtime? How can safety constraints be guaranteed to hold across agent boundaries and execution contexts? How can agent state be preserved and transferred when execution must migrate?

The **Seed Computing paradigm** offers a unifying conceptual framework for these questions. In Seed Computing, every unit of computation — whether a data structure, a reasoning process, an agent, or a workflow — is treated as a *seed*: a self-describing, mobile, composable entity that carries its own identity, capability manifest, behavioral contracts, and execution state. Seeds can germinate (initialize), grow (acquire capability), migrate (transfer runtime context), and compose (form larger computational organisms) in response to environmental signals. This paradigm demands an orchestration layer of commensurate sophistication.

Jasper 2.0 is that orchestration layer. Building upon lessons learned from the Jasper 1.x series — which established the foundational agent registry, basic routing heuristics, and the Aurora Runtime integration surface — Jasper 2.0 represents a ground-up redesign driven by three primary design principles:

1. **Determinism as a first-class property.** Every routing decision in Jasper 2.0 is reproducible, auditable, and mathematically grounded. The Unified Field Equation provides a closed-form expression of routing intent that eliminates ambiguity and enables post-hoc verification of any decision made by the mesh.
2. **Safety as a structural constraint, not an afterthought.** The Invariant Engine is not a monitoring layer bolted onto the orchestration stack; it is a load-bearing architectural component whose type-checked constraints are evaluated at each phase of agent execution. Jasper 2.0 cannot make a routing decision that violates a registered hard invariant.
3. **Mobility as a native capability.** Through the Teleportation Engine and its deep integration with the Aurora Runtime, agent execution contexts are

treated as serializable, migratable, integrity-verifiable objects. Agents are not bound to runtime nodes; they inhabit them transiently, migrating as resource conditions and routing logic dictate.

The remainder of this paper is structured as follows. Section 2 describes the agent mesh architecture and topology. Section 3 formalizes the deterministic routing model and the Unified Field Equation. Section 4 covers capability delegation. Section 5 addresses multi-agent decision coordination. Section 6 presents the Invariant Engine in full. Section 7 provides three worked domain examples. Section 8 details the Aurora Runtime integration and Teleportation Engine. Section 9 develops the mathematical foundations. Section 10 surveys broader applications. Section 11 concludes with a summary of contributions and future directions.

2. Agent Mesh Architecture

The Jasper 2.0 agent mesh is a dynamically configurable, peer-to-peer influenced topology in which agent nodes communicate through a structured event fabric rather than through direct point-to-point channels. Unlike a hierarchical orchestration tree — in which a single controller dispatches work to leaf agents — the Jasper 2.0 mesh supports multi-path routing, adaptive failover, and distributed consensus, making it resilient to individual node failure and capable of scaling to thousands of concurrently active agents.

2.1 Mesh Topology

The mesh is structured as a weighted directed graph $G = (V, E, W)$, where each vertex $v \in V$ represents a registered agent node, each directed edge $e \in E$ represents a communication and delegation pathway between two agents, and $W: E \rightarrow \mathbb{R}$ is a weight function encoding the routing preference between connected nodes. Edge weights are continuously updated by the Jasper 2.0 topology manager based on observed latency, trust tier deltas, and load factor changes reported through the Aurora Runtime event bus. The graph is partitioned into *domain clusters* — subgraphs of agents sharing a primary specialization domain (e.g., medical reasoning, legal

analysis, financial modeling) — connected by *Relay agents* that serve as inter-domain bridges.

2.2 Agent Node Types

Jasper 2.0 defines four canonical agent node types, each with distinct roles in the mesh:

- **Coordinator Agent:** Manages task decomposition, sub-task assignment, and result aggregation for a given workflow instance. Coordinators maintain a global view of in-flight delegations and are responsible for invoking the Role Arbitration Layer (RAL) when conflicts arise. A mesh may contain multiple Coordinators active simultaneously, each managing a distinct workflow.
- **Specialist Agent:** Operates within a bounded domain of expertise (e.g., clinical pharmacology, contract clause extraction, SQL query generation). Specialist agents carry high-fidelity capability vectors and are the primary consumers of routed queries in the UFE model.
- **Relay Agent:** Provides inter-cluster bridging, protocol translation between agents using heterogeneous capability schemas, and buffering of messages across trust-tier boundaries. Relay agents maintain lightweight capability indexes of agents in adjacent clusters and can pre-screen queries before forwarding.
- **Observer Agent:** A passive participant in the mesh, subscribing to event streams for audit, monitoring, compliance telemetry, and anomaly detection. Observers do not participate in task routing but can trigger Invariant Engine escalations upon detecting policy violations in observed event data.

2.3 Agent Profile Schema

Every agent in the Jasper 2.0 mesh is described by a structured **Agent Profile** — a capability manifest that serves as the agent's identity, permission scope, and behavioral contract within the mesh. Agent Profiles are validated at registration time

by the Jasper 2.0 registry service and cryptographically signed to prevent tampering. The schema is defined as follows:

```
AgentProfile { agent_id:      UUID (globally unique, content-addressed)
display_name:   String  node_type:      Enum { Coordinator | Specialist |
Relay | Observer } domain:      String (e.g., "healthcare.clinical",
"legal.contract", "finance.risk") version:      SemVer  capabilities:
Capability[] // ordered by confidence score  Capability {  capability_id: UUID
name:          String  confidence:  Float [0.0, 1.0]  scope:      Enum
{ read | write | execute | delegate }  parameters:  Map
<
String, Type
>
} trust_tier:      Enum { T0_untrusted | T1_basic | T2_verified |
T3_privileged | T4_sovereign } invariant_bindings: InvariantRef[] // references to
registered invariants teleport_enabled: Boolean teleport_constraints:
allowed_target_domains: String[]  state_encryption:  Enum { none | AES256 |
ChaCha20 }  max_migration_latency: Duration  created_at:      Timestamp
registry_signature: Base64(ECDSA-P384) }
```

The *capabilities[]* array is the primary input to the UFE capability match function $C(a,q)$. Each capability entry carries a confidence score reflecting the agent's self-assessed and registry-validated proficiency in executing tasks of that type. The *trust_tier* field maps directly to the $T(a)$ coefficient in the UFE. The *invariant_bindings[]* field links the agent to one or more invariants in the Invariant Engine that govern its behavior. Agents with *teleport_enabled: false* are pinned to their origin runtime node and cannot participate in cross-node migration.

2.4 Mesh Formation Protocol

When Jasper 2.0 initializes — or when a new agent registers — the mesh formation protocol executes in three phases. In **Phase 1 (Discovery)**, the new agent broadcasts a signed Agent Profile advertisement to the Jasper 2.0 registry service. The registry validates the profile signature, checks invariant bindings for consistency, and assigns the agent a topology slot within the appropriate domain cluster. In **Phase 2 (Peering)**, the topology manager computes initial edge weights between the new agent and existing mesh nodes based on capability overlap, domain adjacency, and trust tier compatibility. Edges are only formed between agents whose trust tiers satisfy the mesh's configured trust adjacency policy (e.g., T4_sovereign agents cannot form direct edges with T0_untrusted agents without Relay mediation). In **Phase 3 (Synchronization)**, the new agent receives a compressed topology snapshot and initializes its local routing table. It then subscribes to the Aurora Runtime event bus for topology change notifications, ensuring that subsequent edge weight updates, agent departures, and invariant binding changes are applied incrementally without requiring full re-synchronization.

[Figure 1: Jasper 2.0 Mesh Topology — A weighted directed graph showing three domain clusters (Healthcare, Legal, Finance) connected by Relay agents. Within each cluster, Specialist agents form dense subgraphs with intra-cluster edges weighted by capability overlap scores. Coordinator agents sit at the cluster apex, connected to all Specialists and to the Relay layer. Observer agents appear at the periphery with unidirectional subscription edges (dashed) into the event fabric. Inter-cluster Relay agents maintain bidirectional edges with both adjacent cluster Coordinators, with edge labels showing UFE weight profiles. A global Aurora Runtime event bus is depicted as a horizontal band underlying all nodes, with vertical subscription lines connecting each agent node to the bus.]

Figure 1: Jasper 2.0 Agent Mesh Topology — Domain Clusters, Relay Layer,

3. Deterministic Routing Model

The routing model in Jasper 2.0 is designed around a single governing principle: *every routing decision must be a deterministic, reproducible function of the observable state of the mesh at the time of routing*. This principle ensures that routing decisions can be audited, replayed, and formally verified — properties that are essential in regulated domains such as healthcare and legal services, and that distinguish Jasper 2.0 from probabilistic or heuristic-based orchestration systems.

3.1 Routing as an Optimization Problem

Let A denote the set of active, reachable agent nodes in the mesh, and let q denote an incoming query (a structured task request). The routing function $R: A \times Q \rightarrow A$ must select the agent $a^* \in A$ that maximizes a scalar utility function over the observable agent and query state. This utility function must account for four dimensions of agent fitness:

- **Capability fit:** How well does the agent's capability vector align with the demands of query q ?
- **Trustworthiness:** What is the agent's registered trust tier, and does it satisfy the query's minimum trust requirement?
- **Contextual entropy:** How much uncertainty or instability is present in the agent's current execution context? High entropy indicates a recently migrated, degraded, or overloaded agent that may produce lower-quality results.
- **Load factor:** What is the agent's current task queue depth? High load leads to latency and potential timeout failure.

3.2 The Unified Field Equation (UFE)

The four dimensions above are unified into the **Unified Field Equation**, which defines the routing decision as the argmax of a weighted scalar over the agent set:

Equation 1 — Unified Field Equation (UFE)

$$\mathbf{R}(\mathbf{a}, \mathbf{q}) = \operatorname{argmax}_{\mathbf{a} \in A} [\mathbf{w}_c \cdot \mathbf{C}(\mathbf{a}, \mathbf{q}) + \mathbf{w}_t \cdot \mathbf{T}(\mathbf{a}) - \mathbf{w}_e \cdot \mathbf{E}(\mathbf{a}, \mathbf{t}) + \mathbf{w}_l \cdot \mathbf{L}(\mathbf{a})^{-1}]$$

C(a, q) — Capability match score: the cosine similarity between the agent's capability vector and the query's required capability vector, $\in [0, 1]$

T(a) — Trust tier coefficient: a monotonically increasing scalar mapped from trust_tier $\in \{ T0: 0.0, T1: 0.25, T2: 0.5, T3: 0.75, T4: 1.0 \}$

E(a, t) — Contextual entropy of agent *a* at time *t*: the Shannon entropy of the agent's recent output distribution and state volatility index, $\in [0, 1]$

L(a) — Load factor of agent *a*: normalized current task queue depth relative to agent's declared capacity ceiling, $\in (0, \infty)$; inverted so that lower load yields higher utility

w_c, w_t, w_e, w_l — Tunable weighting coefficients summing to 1.0, configured per mesh deployment profile

The UFE is evaluated over all agents in *A* that (a) satisfy the query's minimum trust tier requirement and (b) have at least one capability entry with a scope covering the query type. Agents failing either hard filter are excluded from the argmax computation before it is evaluated. This pre-filtering step ensures that the UFE's argmax is always taken over a feasible set, preventing the selection of an agent that is structurally incompatible with the query regardless of its UFE score.

3.3 Term Analysis

C(a, q) is computed as the cosine similarity between two embedding vectors: the agent's aggregated capability embedding (constructed as a weighted sum of individual capability embeddings, weighted by confidence scores) and the query's

required capability embedding (extracted by Jasper 2.0's query analyzer module at routing time). This representation allows capability matching to generalize across semantic variations in capability naming — an agent with a capability named "pharmacological interaction analysis" will receive a high C-score for a query requiring "drug-drug interaction checking" even without exact string matching.

T(a) transforms the ordinal trust tier into a continuous scalar. The mapping is intentionally non-linear in its semantic implications: a T2_verified agent is not simply "twice as trustworthy" as a T1_basic agent; rather, the tier boundaries represent categorical assurance milestones (e.g., T3_privileged requires formal behavioral auditing and invariant-binding attestation, while T4_sovereign additionally requires hardware attestation of the underlying runtime node).

E(a, t) is computed by the Aurora Runtime's entropy monitor, which samples the agent's output variance over a configurable rolling window and combines it with a state volatility index derived from recent Teleportation Engine migration events, invariant violation counts, and exception rates. A freshly migrated agent will exhibit transiently elevated entropy as its execution context stabilizes in the new runtime node. The entropy term thus acts as a natural damping mechanism that prevents the UFE from routing high-stakes queries to recently migrated agents before their execution context has re-stabilized.

L(a)⁻¹ is an inverse load penalty. The load factor L(a) is reported by each agent via a heartbeat signal to the Aurora Runtime event bus at configurable intervals (default: 500ms). Agents that fail to report within a configurable timeout window are assigned $L(a) = \infty$, causing their UFE contribution to approach zero and effectively removing them from routing consideration — a safe default in the presence of network partition or agent failure.

3.4 Determinism Guarantees

The UFE produces a deterministic output given identical inputs. Jasper 2.0 achieves global determinism across distributed routing evaluations by (a) timestamping all input state variables with Aurora Runtime logical clock values rather than wall-clock times, (b) snapshotting the agent feasible set atomically at the routing timestamp,

and (c) using IEEE 754 double-precision floating-point arithmetic with deterministic rounding mode throughout the computation. These measures ensure that routing decisions replayed from an audit log will produce identical results, enabling post-hoc verification and regulatory audit compliance.

3.5 Fallback Routing Paths

When the primary routing target (the argmax agent) is unavailable at execution time — due to node failure, invariant violation detected during pre-execution check, or explicit rejection by the target agent — Jasper 2.0 initiates a fallback routing path. The fallback evaluates the UFE over the remaining feasible set with an augmented entropy penalty applied to any agent in the same domain cluster as the failed primary (on the hypothesis that cluster-level issues may be affecting multiple agents). If no feasible agent remains after two fallback levels, the query is escalated to the Coordinator agent for manual decomposition or human escalation.

Query Type	Primary Agent	Fallback Agent	UFE Weight Profile (w_c / w_t / w_e / w_l)
Clinical Decision Support	ClinicalSpecialist-01	GeneralistMedical-02	0.45 / 0.30 / 0.15 / 0.10
Contract Clause Extraction	LegalSpecialist-03	LegalRelay-01	0.40 / 0.35 / 0.15 / 0.10
Real-Time Risk Assessment	FinanceSpecialist-07	FinanceSpecialist-09	0.35 / 0.20 / 0.20 / 0.25
General Knowledge Retrieval	KnowledgeBase-01	KnowledgeBase-02	0.50 / 0.15 / 0.10 / 0.25
Cross-Domain Synthesis	Coordinator-Alpha	Coordinator-Beta	0.30 / 0.40 / 0.20 / 0.10
Compliance Audit Trail	Observer-Compliance-01	Observer-Compliance-02	0.25 / 0.45 / 0.20 / 0.10

Table 1: Sample Routing Decision Table — Query Types, Agent Assignments, and UFE Weight Profiles

4. Capability Delegation

Not all agent tasks can be routed to a single target agent. Complex, multi-step workflows frequently require a primary agent to hand off sub-tasks to other agents — whether because the sub-task lies outside the primary agent's capability domain, because execution parallelism is desired, or because the sub-task requires elevated trust clearance held only by another agent. Jasper 2.0 defines a formal **Capability Delegation Protocol (CDP)** governing these handoffs.

4.1 Capability Tokens

At the core of the CDP is the **Capability Token** — a signed, time-bounded, scope-limited credential that grants a recipient agent the right to execute a specific capability on behalf of the delegating agent. Capability Tokens carry the following fields:

- **token_id:** Globally unique identifier, derived from the delegating agent's agent_id and a monotonic sequence number.
- **delegator_id:** The agent_id of the delegating agent.
- **delegatee_id:** The agent_id of the intended recipient agent.
- **capability_ref:** A reference to the specific capability being delegated, including scope (read/write/execute/delegate).
- **scope_constraints:** Additional constraints on the delegated capability — e.g., maximum result set size, allowed data categories, permitted output formats.
- **valid_from / valid_until:** Time-bounded validity window. All Capability Tokens expire; there are no permanent delegations in Jasper 2.0.
- **delegation_depth:** The maximum number of times this token may be re-delegated (0 = terminal; the delegatee cannot further sub-delegate).

- **invariant_passthrough:** Specifies which invariant bindings from the delegator's profile are inherited by the delegatee for the duration of the delegation.
- **signature:** ECDSA-P384 signature of the token body, verifiable against the delegating agent's registry-registered public key.

4.2 Scoped Delegation Chains

Jasper 2.0 supports **delegation chains** — sequences of capability tokens in which Agent A delegates to Agent B, who further delegates a sub-capability to Agent C, and so on. Chains are governed by two rules. First, the *capability containment rule*: no agent may delegate a capability scope broader than it possesses; Agent B can only delegate capabilities that are a subset of those it received from Agent A. Second, the *depth decay rule*: each re-delegation decrements the `delegation_depth` counter; a token with `delegation_depth = 0` may be consumed but not forwarded. These rules form a lattice of delegation authority that is provably non-escalating: no chain can produce an agent with capabilities exceeding those of the originating Coordinator.

4.3 Revocation Mechanisms

Capability Tokens may be revoked before their expiry in three ways. **Explicit revocation** occurs when the delegating agent issues a signed `RevocationNotice` to the Jasper 2.0 token registry, immediately invalidating the target token and all downstream tokens in its chain. **Invariant-triggered revocation** occurs when the Invariant Engine detects a violation by the delegatee agent; the violating agent's active tokens are suspended pending review, and the delegator is notified. **Trust tier downgrade** occurs when the Jasper 2.0 registry lowers a delegatee agent's `trust_tier` (e.g., following a behavioral audit failure); tokens requiring a higher trust tier than the agent now holds are automatically invalidated. All revocations are recorded in the audit trail with cryptographic proof of the revocation event.

4.4 Delegation Patterns: Atomic and Composite

Jasper 2.0 recognizes two canonical delegation patterns. An **atomic delegation** transfers a single, indivisible capability to a single delegatee for a single task instance. The result is returned directly to the delegator, and the token is consumed upon successful completion. A **composite delegation** transfers a structured bundle of capabilities to a set of delegates in a defined execution order (sequential, parallel, or conditional). The Coordinator agent manages composite delegation through a delegation plan — a directed acyclic graph (DAG) of token issuances and expected result dependencies. The Coordinator's delegation plan is itself registered with the Invariant Engine, which verifies that no invariant binding is violated by any edge in the DAG before the plan is executed.

[Figure 2: Capability Delegation Lifecycle — A temporal sequence diagram showing five phases: (1) Coordinator issues Capability Token to Specialist-A; (2) Specialist-A accepts token, validates signature, checks invariant_passthrough; (3) Specialist-A executes sub-task and optionally issues child token to Specialist-B (delegation_depth decremented); (4) Specialist-B completes execution and returns result to Specialist-A; (5) Specialist-A aggregates result and returns to Coordinator; Coordinator marks token consumed. Revocation pathway shown as dashed red arrow from Invariant Engine to Token Registry, with downstream chain invalidation cascade illustrated.]

Figure 2: Capability Delegation Lifecycle — Token Issuance, Chain Propagation, and Revocation Cascade

5. Multi-Agent Decision Framework

In complex workflows, multiple agents may contribute perspectives, partial results, or competing recommendations toward a final decision or output. Jasper 2.0's **Multi-**

Agent Decision Framework (MADF) defines the protocols by which such multi-agent contributions are coordinated, reconciled, and finalized in a principled manner.

5.1 Consensus Protocols

Jasper 2.0 supports two primary consensus mechanisms for multi-agent agreement:

Quorum-Based Consensus requires a configurable threshold fraction of participating agents (default: $\lceil N/2 \rceil + 1$, where N is the number of agents in the decision group) to produce agreeing outputs — defined as outputs whose semantic embedding similarity exceeds a configurable threshold θ — before a decision is committed. Quorum-based consensus is used for high-stakes, low-latency decisions where speed of convergence is prioritized over weighted authority.

Weighted Trust Voting assigns each participating agent a vote weight equal to its $T(a)$ trust tier coefficient, normalized across the group. Each agent's output is embedded, and the final decision is computed as the trust-weighted centroid of the output embedding space, then mapped to the nearest valid output in the result schema. Weighted trust voting is the default mechanism for regulatory and compliance-sensitive decisions, where higher-trust agents should exert proportionally greater influence over the consensus outcome.

5.2 Conflict Resolution

When consensus mechanisms fail to converge — either because no quorum is reached or because the weighted centroid falls below a minimum semantic coherence threshold — the MADF escalates to the **Role Arbitration Layer (RAL)**. The RAL identifies the Coordinator agent with the highest UFE score relative to the conflicted decision domain and grants it arbitration authority. The arbitrating Coordinator may (a) select the single agent output with the highest individual UFE score as the authoritative result, (b) decompose the conflict into sub-questions and initiate a new routing cycle for each, or (c) escalate to a human-in-the-loop endpoint if the conflict is flagged as unresolvable within the system's confidence bounds. All RAL arbitration decisions are logged with full provenance.

5.3 Temporal Ordering

Multi-agent responses are ordered using Aurora Runtime logical timestamps. When agents respond to a broadcast query asynchronously, their responses are received by the Coordinator in physical arrival order but processed in logical clock order — ensuring that causally later responses are always considered after the events they depend on, regardless of network jitter. Responses arriving after a configurable deadline window are discarded and the discarding agent's $L(a)$ load factor is penalized for the subsequent routing cycle.

5.4 Role Arbitration Layer (RAL)

The RAL is a dedicated subsystem of Jasper 2.0 that mediates agent role assignments in dynamic workflow scenarios. Beyond conflict resolution, the RAL handles: (a) *role promotion*, in which a Specialist agent is temporarily elevated to Coordinator authority when the primary Coordinator is unavailable; (b) *role splitting*, in which a single Coordinator's decision domain is partitioned across two Coordinators when mesh load exceeds thresholds; and (c) *role merger*, in which two Coordinators managing independent workflows are combined when their task graphs are determined to share a dependency. All RAL role changes are propagated to affected agents via the Aurora Runtime event bus and recorded in the mesh topology log.

Decision Mode	Description	Trigger Conditions	Consensus Mechanism
Autonomous	Single agent produces and commits result without peer consultation	Single feasible agent; query confidence ≥ 0.90 ; no mandatory review invariants	None (single agent)
Assisted	Primary agent produces result with secondary validation from one peer agent	Query confidence 0.70–0.89; at least one soft review invariant binding	Bilateral agreement (2-agent quorum)
Delegated	Primary agent delegates sub-	Multi-domain query; composite	Weighted trust voting over

Decision Mode	Description	Trigger Conditions	Consensus Mechanism
	tasks to specialists; Coordinator aggregates	capability requirement; task DAG depth ≥ 2	specialist outputs
Escalated	No automated consensus reached; decision referred to human-in-the-loop or senior Coordinator	Consensus failure; hard invariant conflict; query confidence < 0.50 ; RAL arbitration inconclusive	Human override or sovereign-tier Coordinator arbitration

Table 2: Multi-Agent Decision Modes — Descriptions, Triggers, and Consensus Mechanisms

6. Safety & Compliance Logic

Safety is not an optional property of the Jasper 2.0 mesh — it is a structural invariant of its architecture. The **Invariant Engine** is the subsystem responsible for defining, evaluating, and enforcing typed behavioral constraints across all agents, delegations, and routing decisions in the mesh. Its design is informed by formal methods principles: invariants are verifiable propositions about agent and system state, not ad hoc policy rules.

6.1 Invariant Types

Jasper 2.0 distinguishes two classes of invariants:

Hard Invariants are non-negotiable constraints whose violation prevents the associated action from executing. A routing decision that would violate a hard invariant is blocked at the UFE evaluation stage — the violating agent is removed from the feasible set before argmax is evaluated. A delegation that would violate a hard invariant causes the Capability Token to be rejected before issuance. Hard invariants are typed as boolean predicates over the system state at evaluation time, and their evaluation must terminate in bounded time (the Invariant Engine enforces a configurable maximum evaluation timeout, defaulting to 50ms).

Soft Invariants express preferences or guidelines that should be satisfied where possible but may be overridden under defined conditions. Soft invariant violations do not block execution; they produce a *compliance warning* event on the Aurora Runtime event bus, increment the violating agent's entropy score $E(a,t)$, and are recorded in the audit trail. Repeated soft invariant violations by the same agent may trigger an automatic trust tier review.

6.2 Safety Check Pipeline

The Invariant Engine evaluates constraints at three phases of agent execution:

- **Pre-Execution:** Before a routed query or delegated capability reaches the target agent's execution context, all hard invariants bound to both the agent and the query type are evaluated. If any hard invariant evaluates to false, the routing or delegation is rejected and the fallback path is invoked. Soft invariants are also evaluated; warnings are recorded but execution proceeds.
- **Mid-Execution:** For long-running tasks (duration exceeding a configurable threshold, default: 5 seconds of wall-clock time within the Aurora Runtime), the Invariant Engine subscribes to execution progress events from the agent's Aurora Runtime context. It re-evaluates a subset of hard invariants — specifically those tagged as *continuous* rather than *point-in-time* — at each progress event. A mid-execution hard invariant violation triggers immediate task suspension, state snapshot, and escalation to the Coordinator.
- **Post-Execution:** After an agent produces a result, all output invariants are evaluated against the result payload before it is returned to the Coordinator or committed to the delegation chain. Output invariants may check for data sensitivity classification, result size bounds, required output field presence, and format compliance.

6.3 External Compliance Framework Integration

Jasper 2.0 provides an **External Policy Adapter (EPA)** interface that allows invariants to delegate evaluation to external compliance frameworks — such as HIPAA

policy engines, GDPR data residency validators, or enterprise-specific policy-as-code systems (e.g., Open Policy Agent). When an invariant's evaluation logic references an EPA, the Invariant Engine dispatches a signed, scoped evaluation request to the external framework and awaits a boolean response within the evaluation timeout window. EPA calls are cached with a configurable TTL to avoid redundant external calls for identical query/agent state combinations.

6.4 Audit Trail

Every Invariant Engine evaluation — whether passing or failing — is appended to a tamper-evident audit log maintained by the Aurora Runtime. Log entries are chained using SHA-3-256 Merkle tree construction, ensuring that any retrospective modification of the log is detectable. The audit log records the invariant ID, evaluation timestamp (Aurora logical clock + wall clock), input state snapshot hash, evaluation result, and (for EPA calls) the external framework response. Audit logs are exportable in JSONL, XML, and NDJSON formats for integration with SIEM systems.

6.5 Sample Invariant Definition

The following YAML-style pseudocode illustrates a representative hard invariant binding for a healthcare domain Specialist agent, enforcing that the agent may only return clinical recommendations for de-identified patient data:

```
invariant: invariant_id: "inv-hc-001" name:
  "PatientDataDeidentificationRequirement" type: HARD evaluation_phase:
  [PRE_EXECUTION, POST_EXECUTION] continuous: false description: >
  Clinical recommendation agents MUST NOT process or return data containing
  direct patient identifiers (name, DOB, MRN, SSN, address). All input and output
  payloads must pass de-identification verification before processing. bound_to:
  - domain: "healthcare.clinical" - node_types: [Specialist, Coordinator]
  pre_execution_predicate: validator: "deidentification.check_input" parameters:
```

```
input_field: "query.payload"    identifier_classes: [NAME, DOB, MRN, SSN,
ADDRESS, PHONE]    threshold: 0.0 # zero tolerance for identified data
post_execution_predicate:    validator: "deidentification.check_output"
parameters:    output_field: "result.payload"    identifier_classes: [NAME, DOB,
MRN, SSN, ADDRESS, PHONE]    threshold: 0.0    on_violation:    action:
BLOCK_AND_ESCALATE    escalation_target: "Coordinator-Healthcare-01"
audit_severity: CRITICAL    notify: ["compliance-officer@seed-health.org"]
external_policy_adapter:    framework: OpenPolicyAgent    policy_bundle:
"hipaa_phi_filter_v3.1"    evaluation_timeout_ms: 40
```

6.6 Escalation Protocols

When a hard invariant violation occurs and the fallback routing path also fails invariant evaluation, Jasper 2.0 initiates a structured escalation protocol. The escalation proceeds through three levels: (Level 1) automatic re-routing to the highest-trust available agent in the domain with a penalty applied to the violating agent's entropy score; (Level 2) Coordinator arbitration with human notification dispatched to registered compliance contacts; (Level 3) full workflow suspension with cryptographic sealing of the workflow state snapshot, pending manual review and re-authorization by a T4_sovereign-tier agent or human operator. The escalation level reached for each incident is recorded in the audit trail as a CRITICAL severity event.

7. Domain-Specific Orchestration Examples

The following three examples illustrate the end-to-end operation of Jasper 2.0 across representative deployment domains. Each example describes the agent profile configuration, routing path as determined by the UFE, invariants applied, and the resulting workflow outcome.

7.1 Healthcare: Multi-Agent Clinical Decision Support

Scenario: A hospital information system submits a query requesting a clinical decision support recommendation for a patient presenting with polypharmacy complications. The query carries a minimum trust tier requirement of T3_privileged and a hard invariant requirement of inv-hc-001 (PatientDataDeidentificationRequirement). All patient data has been pre-processed by the hospital's de-identification pipeline before submission to Jasper 2.0.

Agent Profile Setup: The healthcare cluster comprises three Specialist agents: ClinicalPharmacology-01 (trust_tier: T3, capabilities: [pharmacological_interaction_analysis: 0.95, polypharmacy_risk_assessment: 0.92]), ClinicalDiagnosis-02 (trust_tier: T3, capabilities: [differential_diagnosis: 0.90, medication_contraindication: 0.85]), and GeneralistMedical-03 (trust_tier: T2, capabilities: [clinical_summarization: 0.80]). A Coordinator-Healthcare-01 (trust_tier: T4) manages the cluster. All three Specialists have inv-hc-001 bound to their profiles.

Routing Path: The UFE is evaluated with the weight profile (w_c : 0.45, w_t : 0.30, w_e : 0.15, w_l : 0.10). ClinicalPharmacology-01 achieves the highest UFE score (0.887) on the strength of its high capability match ($C = 0.94$) and trust tier ($T = 0.75$), with low current entropy ($E = 0.12$) and moderate load ($L = 0.60$). ClinicalPharmacology-01 is selected as primary. A composite delegation is initiated: ClinicalPharmacology-01 delegates a "medication_contraindication" sub-query to ClinicalDiagnosis-02 via a Capability Token with delegation_depth: 0 and a validity window of 30 seconds.

Invariants Applied: inv-hc-001 is evaluated at pre-execution for both ClinicalPharmacology-01 and ClinicalDiagnosis-02. Both pass (the input payload contains no direct identifiers). Post-execution, both output payloads are verified clean. A secondary soft invariant (inv-hc-soft-004: PreferEvidence-BasedGuidelines) is evaluated; ClinicalDiagnosis-02's output references a clinical guideline with a confidence score above threshold, so no warning is issued.

Outcome: Coordinator-Healthcare-01 aggregates the pharmacological risk assessment from ClinicalPharmacology-01 with the contraindication analysis from ClinicalDiagnosis-02 using weighted trust voting (both agents at T3, equal weights).

The synthesized recommendation is returned to the hospital system with full audit trail reference and an attached invariant evaluation certificate. Total round-trip latency: 1.4 seconds, within the healthcare SLA of 2.0 seconds.

7.2 Legal & Compliance: Contract Analysis Delegation Chain

Scenario: An enterprise legal operations platform submits a 200-page commercial contract for multi-axis analysis: clause extraction, risk flagging, jurisdiction determination, and comparison against a reference master services agreement (MSA). The query requires T2_verified minimum trust and carries a data residency invariant (inv-legal-003: DataResidencyEU) specifying that all intermediate data must remain within EU-designated Aurora Runtime nodes.

Agent Profile Setup: The legal cluster contains: ContractExtraction-01 (T3, capabilities: [clause_extraction: 0.96, obligation_mapping: 0.91]), RiskAnalysis-02 (T2, capabilities: [contractual_risk_flagging: 0.88, deviation_from_standard: 0.85]), JurisdictionClassifier-03 (T3, capabilities: [governing_law_identification: 0.93, jurisdiction_conflict_detection: 0.89]), and Coordinator-Legal-01 (T4). inv-legal-003 is bound to the entire cluster.

Routing Path: Coordinator-Legal-01 decomposes the query into four sub-tasks using a composite delegation plan. ContractExtraction-01 is assigned clause extraction (UFE score: 0.912); RiskAnalysis-02 receives a Capability Token conditioned on the output of ContractExtraction-01 (sequential dependency edge in the delegation DAG); JurisdictionClassifier-03 runs in parallel with RiskAnalysis-02. The MSA comparison is handled by RiskAnalysis-02 as an atomic sub-delegation from ContractExtraction-01, at delegation_depth: 1. All Capability Tokens specify EU-resident Aurora Runtime node constraints via their scope_constraints field.

Invariants Applied: inv-legal-003 is evaluated pre-execution for all four agents; the Aurora Runtime node affinity system confirms all assigned nodes carry the EU data residency attestation. A mid-execution check on RiskAnalysis-02 at the 8-second mark confirms the agent remains on an EU-resident node (no migration has occurred). Post-execution, output payloads are checked for inadvertent PII inclusion under inv-legal-004 (a soft invariant); no violations are detected.

Outcome: Coordinator-Legal-01 produces a structured legal analysis report comprising 47 extracted clauses, 12 risk flags (3 high-severity), 2 jurisdiction conflicts, and a 94-point MSA deviation score. The full audit trail, including invariant evaluation certificates and Capability Token chain records, is attached and exported to the enterprise legal platform's compliance repository. Total latency: 22.3 seconds; within the legal operations SLA of 60 seconds for large-document analysis.

7.3 Enterprise Automation: Cross-System Workflow Orchestration

Scenario: An enterprise IT orchestration platform uses Jasper 2.0 to manage a cross-system employee onboarding workflow spanning six backend systems: HR information system (HRIS), identity and access management (IAM), IT asset provisioning, benefits enrollment, training assignment, and communications platform configuration. The workflow must complete within a 4-hour SLA and must not provision access to any system before identity verification is confirmed by the IAM agent.

Agent Profile Setup: Six Specialist agents, one per target system, each with domain-specific capabilities (e.g., IAMSpecialist-01: [identity_verification: 0.97, access_provisioning: 0.95]). A Coordinator-Enterprise-01 (T4) manages the workflow. A hard invariant (inv-ent-007: IdentityVerificationGating) is bound to all non-IAM Specialist agents, specifying that access provisioning capabilities may only be executed after receiving a signed completion token from IAMSpecialist-01.

Routing Path: Coordinator-Enterprise-01 constructs a delegation DAG in which IAMSpecialist-01 is the root node. Upon IAM completion, it issues a signed completion token that unlocks the access_provisioning capabilities of the remaining five Specialist agents. The UFE is evaluated for each subsequent agent assignment with a weight profile emphasizing load factor (w_l: 0.25) to ensure the workflow remains within the 4-hour SLA under peak provisioning load.

Invariants Applied: inv-ent-007 is evaluated pre-execution for all non-IAM agents; those invoked before the IAM completion token is available fail the pre-execution check and are held in a pending queue until the token arrives. A soft invariant (inv-ent-soft-012: ProvisioningOrderPreference) recommends that IT asset provisioning occur

before benefits enrollment; this preference is respected in the delegation DAG ordering.

Outcome: The full onboarding workflow completes in 47 minutes, well within the 4-hour SLA. All six Specialist agents report successful task completion. The Coordinator commits the workflow outcome to the enterprise audit log with full delegation chain provenance. The employee's new system access is active and verified across all six platforms simultaneously upon workflow completion.

8. Integration with Aurora Runtime

Jasper 2.0 is not a standalone orchestration system — it is designed as a first-class citizen of the **Aurora Runtime**, the execution substrate of the Seed Computing paradigm. Aurora Runtime provides the foundational services upon which Jasper 2.0 depends: agent lifecycle management, logical clock synchronization, event bus infrastructure, state snapshot facilities, and the Teleportation Engine for agent migration. The integration between Jasper 2.0 and Aurora Runtime is deep, bidirectional, and semantically rich.

8.1 Aurora Runtime Overview

Aurora Runtime is a distributed, polyglot execution environment that manages the lifecycle of seed entities — including Jasper 2.0 agent seeds — across a network of runtime nodes. Each Aurora Runtime node hosts an isolation-guaranteed execution sandbox for each resident seed, with configurable resource quotas (CPU, memory, network bandwidth, storage I/O). Aurora Runtime nodes are organized into a federated cluster managed by an Aurora Cluster Controller, which maintains the authoritative node membership list, health status, and resource availability map used by the Jasper 2.0 UFE computation. Aurora Runtime exposes its services to Jasper 2.0 through a well-defined API surface described below.

8.2 Aurora-Jasper API Surface

The Aurora-Jasper integration API comprises three categories of primitives:

- **Lifecycle Hooks:** *OnAgentRegister*, *OnAgentDeregister*, *OnAgentSuspend*, *OnAgentResume*, *OnAgentMigrate*. These hooks allow Jasper 2.0 to update its topology graph, recompute edge weights, and trigger fallback routing in response to agent lifecycle state changes within the Aurora Runtime.
- **Event Bus:** Aurora Runtime's event bus is a publish-subscribe fabric supporting topic-based routing with per-topic retention policies. Jasper 2.0 publishes routing decisions, delegation events, invariant evaluations, and topology changes to the bus. Observer agents subscribe to relevant topics. The Invariant Engine subscribes to agent execution progress events for mid-execution safety checks. The event bus guarantees at-least-once delivery with idempotency keys.
- **State Snapshots:** On demand or at configurable intervals, Aurora Runtime produces a cryptographically signed snapshot of an agent's full execution context: heap contents, active Capability Token set, pending query queue, outgoing delegation state, and Invariant Engine binding state. Snapshots are the unit of transfer for the Teleportation Engine and the unit of recovery for agent restart after node failure.

8.3 Agent Registration in Aurora

When Jasper 2.0 registers a new agent with the mesh, it simultaneously registers the agent's execution context with the Aurora Runtime Cluster Controller. This registration includes: (a) the agent's resource quota request, (b) the node affinity constraints derived from the agent's Agent Profile (e.g., data residency requirements, hardware attestation requirements for T4_sovereign agents), and (c) the set of Aurora Runtime event topics the agent should be subscribed to. The Cluster Controller selects an appropriate runtime node satisfying the affinity constraints, initializes the agent's execution sandbox, and returns a runtime handle to Jasper 2.0 for use in subsequent lifecycle operations. Agent registration is atomic: if any step fails, the registration is rolled back entirely, and the agent does not appear in the Jasper 2.0 mesh.

8.4 The Teleportation Engine

The **Teleportation Engine** is the subsystem of Aurora Runtime responsible for migrating agent execution contexts — and their associated Jasper 2.0 orchestration state — between runtime nodes. The term "teleportation" reflects the Seed Computing paradigm's treatment of agent state as a living, mobile entity: a seed does not *copy* to a new node; it *migrates*, with its origin context atomically destroyed upon successful establishment at the destination. The Teleportation Engine executes the following protocol:

Phase 1 — Pre-Migration Notification: Jasper 2.0 notifies the Invariant Engine and Coordinator of the pending migration. The Coordinator suspends new capability token issuances to the migrating agent. In-flight delegations are allowed to complete or are transferred to a standby agent per the fallback routing policy.

Phase 2 — State Serialization: Aurora Runtime produces a full state snapshot of the migrating agent's execution context. The serialization format is Aurora Object Stream (AOS): a length-prefixed, schema-versioned binary format supporting incremental deserialization, forward and backward compatibility across Aurora Runtime versions, and optional ChaCha20 encryption for snapshots crossing trust tier boundaries.

Phase 3 — Integrity Verification: The serialized snapshot is hashed using SHA-3-256 and signed by the origin Aurora Runtime node's attestation key. The destination node verifies the signature and hash before accepting the snapshot. If verification fails, the migration is aborted and the agent remains on the origin node. Integrity verification is non-optional and cannot be bypassed.

Phase 4 — Context Establishment: The destination Aurora Runtime node deserializes the snapshot into a new execution sandbox, verifying schema version compatibility. The agent's Capability Token set and Invariant Engine binding state are re-registered with the local Jasper 2.0 mesh node serving the destination runtime. The agent's entropy score $E(a,t)$ is incremented to reflect migration-induced context volatility.

Phase 5 — Origin Teardown and Handoff: The origin node atomically destroys the agent's execution sandbox and emits an *OnAgentMigrate* event to the Aurora Runtime event bus. Jasper 2.0 updates its topology graph to reflect the agent's new runtime node assignment, recomputes affected edge weights, and resumes capability token issuance to the agent.

Latency Characteristics: Teleportation Engine migration latency is dominated by the state serialization and network transfer phases. For agents with execution context sizes below 256MB (the target ceiling for Jasper 2.0 Specialist agents), end-to-end migration latency within a single Aurora Runtime datacenter cluster is targeted at under 800ms for unencrypted migrations and under 1.2 seconds for ChaCha20-encrypted cross-trust-boundary migrations. Cross-datacenter migrations (e.g., EU to US-East) operate under a separate latency SLA of 3.5 seconds, accounting for wide-area network transit.

[Figure 3: Aurora-Jasper Teleportation Sequence — A vertical swimlane diagram with lanes for: Coordinator (Jasper 2.0), Invariant Engine, Origin Aurora Node, Destination Aurora Node, and Aurora Event Bus. The sequence shows: (1) Coordinator initiates migration request; (2) Invariant Engine grants migration approval (pre-condition check); (3) Origin Node serializes agent context to AOS format; (4) Origin Node transmits AOS blob to Destination Node with SHA-3-256 signature; (5) Destination Node verifies signature and deserializes; (6) Destination Node emits AgentMigrate event to Event Bus; (7) Coordinator receives event, updates topology; (8) Origin Node tears down sandbox. Failure path shown as dashed arrow from step (5) back to Coordinator with MIGRATION_FAILED event, triggering fallback routing.]

Figure 3: Teleportation Engine Migration Sequence — Origin Serialization, Integrity Verification, and Destination Establishment

9. Mathematical Foundations

This section provides a more rigorous treatment of the formal model underlying Jasper 2.0's routing and decision systems, establishing the mathematical structures within which the UFE, capability matching, and dynamic rebalancing operate.

9.1 Agent Capability Space

Let C denote the **Agent Capability Space**, defined as a real-valued vector space of dimension d , where d is the dimensionality of the capability embedding model used by Jasper 2.0's query analyzer. Each agent a is associated with a capability vector $\mathbf{c}_a \in C$, computed as the confidence-weighted sum of the individual capability embeddings in the agent's capability array:

Equation 2 — Agent Capability Vector

$$\mathbf{c}_a = \sum_i \text{conf}_i \cdot \text{embed}(\text{cap}_i) / \sum_i \text{conf}_i$$

where cap_i is the i -th capability in the agent's profile, conf_i is its confidence score, and $\text{embed}(\cdot)$ is the capability embedding function mapping capability names to $\mathbb{R}^d$

Similarly, each query q is associated with a required capability vector $\mathbf{c}_q \in C$ extracted by the query analyzer. The capability match score $C(a, q)$ is then defined as the cosine similarity in C :

Equation 3 — Capability Match Score

$$C(a, q) = (\mathbf{c}_a \cdot \mathbf{c}_q) / (\|\mathbf{c}_a\| \cdot \|\mathbf{c}_q\|)$$

This formulation places $C(a, q) \in [-1, 1]$ in general; however, because capability embeddings are computed from non-negative confidence-weighted sums over a non-

negative embedding space, $C(a,q) \in [0, 1]$ for all valid agents and queries in practice. The embedding space C is an inner product space under the standard dot product, and the cosine similarity metric is well-defined and bounded throughout.

9.2 The UFE as a Convergent Optimization Problem

The UFE routing decision $R(a,q)$ is a maximization over the finite feasible agent set $A_{\text{feas}} \subseteq A$ (after pre-filtering). Because A_{feas} is finite and non-empty (guaranteed by the fallback escalation protocol, which ensures at least one agent — the Coordinator — is always feasible for any query), and because the UFE objective function is a sum of bounded, continuous terms over a finite discrete set, the argmax is always well-defined and attained. The UFE therefore does not suffer from the non-convergence problems associated with continuous optimization over unbounded domains.

Convergence under dynamic agent state: In a dynamic mesh where agent state (entropy, load) changes continuously, the UFE's selected agent may change between successive evaluations of the same query. However, we can show that the UFE is *Lyapunov stable* under standard conditions: if the mesh is in a state where agent A is the argmax, and A's load increases monotonically (a common scenario under sustained traffic), then eventually agent B becomes the argmax, and traffic is redistributed. This redistribution reduces B's $L(a)$ term, and in the absence of new external load, the system converges to a balanced state in which no single redistribution of a query would improve the global UFE sum. This is equivalent to a Nash equilibrium condition for the routing game, which is known to exist and be unique for strictly concave utility functions — a condition satisfied by the UFE's $L(a)^{-1}$ term.

9.3 The Entropy Gradient and Dynamic Rebalancing

The contextual entropy term $E(a,t)$ is not static; it evolves continuously as a function of the agent's operational history. Jasper 2.0 introduces the **Entropy Gradient** $\nabla E(a,t)$ to characterize the rate of entropy change and to enable proactive rebalancing:

Equation 4 — Entropy Gradient

$$\nabla E(a, t) = \partial E(a, t) / \partial t = \alpha \cdot \sigma^2_{\text{output}}(a, t) + \beta \cdot v_{\text{state}}(a, t) - \gamma \cdot \tau(a, t)$$

$\sigma^2_{\text{output}}(a, t)$ — variance of agent a 's output embeddings over a rolling window ending at t

$v_{\text{state}}(a, t)$ — state volatility index: rate of Aurora Runtime state snapshot divergence from baseline

$\tau(a, t)$ — stabilization factor: time elapsed since last migration or invariant violation event, normalized by characteristic stabilization time constant

α, β, γ — positive scalar coefficients ($\alpha + \beta > \gamma$ to ensure entropy is non-negative)

When $\nabla E(a, t) > 0$ (entropy increasing), Jasper 2.0's topology manager proactively reduces the edge weight of edges incident to agent a , softly discouraging new routing to that agent before it enters high-entropy territory. When $\nabla E(a, t) < 0$ (entropy decreasing, agent stabilizing), edge weights are progressively restored, allowing the agent to return to full routing participation. This gradient-driven edge weight modulation is the mechanism by which the mesh dynamically self-balances without requiring a central control loop.

9.4 Complexity Analysis

The time complexity of a single UFE evaluation over a feasible agent set of size $|A_{\text{feas}}| = n$ is $O(n)$ — a linear scan over the feasible set computing the objective function value for each agent. However, the practical routing pipeline includes the pre-filtering step (which requires checking trust tier and capability scope constraints for all registered agents in the mesh) and the capability match computation (which requires embedding-space dot product operations of dimension d). The full routing complexity is therefore $O(|A| \cdot d)$ for the filtering and match computation phase, followed by $O(n)$ for the UFE argmax.

For mesh routing under concurrent load (multiple simultaneous queries), Jasper 2.0 parallelizes UFE evaluations across queries using Aurora Runtime worker threads. The dominant bottleneck is the topology graph maintenance under high agent churn: edge weight recomputation following a topology change event requires updating the weights of all edges incident to the changed node, which in the worst case is $O(n)$ for a fully connected subgraph. Jasper 2.0 mitigates this by maintaining a sparse topology graph (maximum configurable node degree k), reducing edge weight recomputation to $O(k)$ per topology change event. Under the sparse topology constraint, the overall mesh routing complexity is bounded by **$O(n \log n)$** amortized across concurrent routing requests, where the $\log n$ factor arises from the priority queue used to order agents by UFE score during argmax computation.

10. Applications

The Jasper 2.0 architecture is a general-purpose agent orchestration system; while the worked examples in Section 7 illustrate three specific deployment scenarios, the system's capabilities extend to a broad range of domains wherever complex, multi-step, safety-sensitive, and scale-demanding agent coordination is required.

10.1 Autonomous Research Systems

In large-scale scientific research environments, Jasper 2.0 enables the orchestration of autonomous research agent meshes in which Specialist agents handle literature retrieval, hypothesis generation, experimental design, data analysis, and manuscript drafting — each operating within strictly defined capability domains and delegating cross-domain sub-tasks through the CDP. The Invariant Engine enforces methodological constraints (e.g., "experimental conclusions may only be drawn from statistically significant results") at the post-execution phase, preventing unsupported claims from propagating through the delegation chain. The Teleportation Engine enables long-running research workflows to migrate across Aurora Runtime nodes in response to computational resource availability, without interrupting the workflow's logical continuity.

10.2 Real-Time Financial Risk Orchestration

Financial services firms operating algorithmic trading, credit risk assessment, and regulatory reporting systems require sub-second decision latency combined with strict auditability and compliance. Jasper 2.0's deterministic routing model and $O(n \log n)$ complexity bound make it suitable for real-time financial orchestration at throughputs exceeding 10,000 queries per second under appropriate Aurora Runtime node provisioning. Hard invariants enforce position limits, regulatory reporting obligations, and circuit breaker conditions — ensuring that no agent in the mesh can produce a trading signal or risk assessment that violates registered compliance constraints. The weighted trust voting mechanism provides a natural model for aggregating signals from multiple risk models of differing confidence levels into a single authoritative risk score.

10.3 Adaptive Education Platforms

Personalized learning systems can deploy Jasper 2.0 to orchestrate meshes of pedagogical Specialist agents — each optimized for a specific subject domain, learning modality, or student proficiency tier — routing individual learner interactions to the most appropriate pedagogical agent in real time. The UFE's capability match function naturally accommodates the learning context: the query embedding encodes not just the subject matter but also the learner's demonstrated proficiency level and preferred interaction style, enabling the routing function to select agents whose pedagogical capabilities align with the learner's current needs. Soft invariants enforce pedagogical best practices (e.g., "scaffolding constraints before procedural practice") without preventing adaptation when the learner's state warrants deviation.

10.4 Government AI Coordination Layers

Government agencies deploying AI systems across multiple departments, classification levels, and jurisdictions face coordination challenges that Jasper 2.0 is specifically designed to address. The trust tier model maps directly to security clearance levels; the data residency invariants enforce jurisdictional data sovereignty requirements; the Relay agent architecture provides controlled inter-domain

communication across classification boundaries; and the tamper-evident audit trail satisfies government records management and oversight requirements. The Teleportation Engine's cross-domain migration constraints (specified in the Agent Profile's `teleport_constraints.allowed_target_domains` field) ensure that agents handling classified information cannot be migrated to unclassified runtime nodes.

10.5 Smart Infrastructure Management

Urban smart infrastructure systems — encompassing traffic management, utility grid control, emergency response coordination, and environmental monitoring — require real-time multi-system orchestration with strong safety guarantees. Jasper 2.0's hard invariant system can encode physical safety constraints (e.g., "traffic signal agents may not create conflicting green phases on intersecting lanes") that are enforced at the pre-execution phase before any actuation command is issued. The mesh architecture's resilience to individual node failure — through fallback routing and Teleportation Engine migration — ensures that critical infrastructure control remains operational even in the presence of individual runtime node failures. The Observer agent pattern provides a natural fit for continuous monitoring of infrastructure state without interfering with control loop latency.

11. Conclusion

This paper has presented **Jasper 2.0**, a comprehensive agent mesh and orchestration system engineered for deployment within the Seed Computing paradigm. The system's principal contributions are: (1) the **Unified Field Equation**, a mathematically grounded, deterministic routing decision function that integrates capability matching, trust tier enforcement, contextual entropy, and load balancing into a single, auditable scalar; (2) the **Capability Delegation Protocol**, a formally specified mechanism for scoped, revocable, non-escalating capability transfer across agent boundaries; (3) the **Multi-Agent Decision Framework**, providing quorum-based and trust-weighted consensus protocols with structured conflict resolution via the Role Arbitration Layer; (4) the **Invariant Engine**, a typed, formally verifiable constraint-enforcement subsystem providing pre-, mid-, and post-execution safety

guarantees; and (5) deep integration with the **Aurora Runtime** through the Teleportation Engine, enabling live agent state migration with full integrity verification.

Together, these components realize the core promise of the Seed Computing paradigm: that intelligence, computation, and state can be treated as living, mobile, composable entities — seeds — that adapt, migrate, and cooperate in response to environmental conditions without sacrificing safety, auditability, or deterministic behavior. Jasper 2.0 demonstrates that these properties are not in tension; they are mutually reinforcing when the orchestration layer is designed with formal rigor from the outset.

Several directions for future work present themselves as natural extensions of the Jasper 2.0 architecture. **Self-modifying agent profiles** — in which agents update their own capability vectors and invariant bindings at runtime based on observed performance — introduce challenges for UFE stability and invariant monotonicity that require new formal guarantees. **Quantum-ready routing** will be required as Aurora Runtime nodes operating quantum processing units (QPUs) enter the mesh; the UFE's capability embedding model and argmax computation must be extended to handle superposition-valued capability scores. **Cross-paradigm federation** — enabling Jasper 2.0 meshes to interoperate with agent systems operating under non-Seed Computing orchestration models — requires a standardized inter-paradigm capability exchange protocol that preserves Jasper 2.0's trust and invariant guarantees at federation boundaries. These directions chart the course for the Jasper 3.x research agenda within the Seed Computing Research Division.

References

4. [1] M. Wooldridge and N. R. Jennings, "Intelligent agents: Theory and practice," *The Knowledge Engineering Review*, vol. 10, no. 2, pp. 115–152, 1995.
5. [2] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Hoboken, NJ: Pearson, 2020.

6. [3] L. Panait and S. Luke, "Cooperative multi-agent learning: The state of the art," *Autonomous Agents and Multi-Agent Systems*, vol. 11, no. 3, pp. 387–434, 2005.
7. [4] F. Belfrage, G. Caire, and D. Greenwood, *Developing Multi-Agent Systems with JADE*. Chichester, UK: Wiley, 2007.
8. [5] R. Brooks, "A robust layered control system for a mobile robot," *IEEE Journal on Robotics and Automation*, vol. 2, no. 1, pp. 14–23, 1986.
9. [6] T. Sandholm and V. R. Lesser, "Coalitions among computationally bounded agents," *Artificial Intelligence*, vol. 94, no. 1–2, pp. 99–137, 1997.
- 10.[7] A. S. Tanenbaum and M. Van Steen, *Distributed Systems: Principles and Paradigms*, 3rd ed. Upper Saddle River, NJ: Prentice Hall, 2017.
- 11.[8] L. Lamport, "Time, clocks, and the ordering of events in a distributed system," *Communications of the ACM*, vol. 21, no. 7, pp. 558–565, 1978.
- 12.[9] S. Seshia, D. Sadigh, and A. Dragan, "Toward verified artificial intelligence," *Communications of the ACM*, vol. 65, no. 7, pp. 46–55, 2022.
- 13.[10] A. Pnueli, "The temporal logic of programs," in *Proceedings of the 18th Annual Symposium on Foundations of Computer Science (FOCS)*, pp. 46–57, 1977.
- 14.[11] J. Ferber, *Multi-Agent Systems: An Introduction to Distributed Artificial Intelligence*. Harlow, UK: Addison-Wesley, 1999.
- 15.[12] Z. Allen-Zhu, Y. Li, and Z. Song, "A convergence theory for deep learning via over-parameterization," in *Proceedings of the 36th International Conference on Machine Learning (ICML)*, pp. 242–252, 2019.
- 16.[13] Open Policy Agent Project, "OPA: The Open Policy Agent," *Cloud Native Computing Foundation Technical Documentation*, v0.63.0, 2025. [Online]. Available: <https://www.openpolicyagent.org>
- 17.[14] Leon, "Aurora Runtime: Seed Lifecycle Management and Execution Substrate Architecture," Seed Computing Technical Report SC-TR-2025-003, Seed Computing Research Division, 2025.

- 18.[15] Leon, "Invariant Engine: Formal Constraint Verification for Agent Mesh Systems," Seed Computing Technical Report SC-TR-2025-011, Seed Computing Research Division, 2025.
- 19.[16] Leon, "The Seed Computing Paradigm: Principles, Architecture, and Formal Foundations," Seed Computing Technical Report SC-TR-2024-001, Seed Computing Research Division, 2024.

Jasper 2.0 Whitepaper

|

SC-WP-2026-007

|

Version 2.0.1

|

Classification: Public Release

|

©

2026 Seed Computing Research Division